

AAP11 RESTful APIs in Python

V2.1

Überblick

Eine API (Application Programming Interface) ist eine Black Box, an die andere Programme Anfragen (requests) stellen können, um Daten zu bekommen oder Funktionen zu nutzen.

Eine REST API ist eine API, die den Designprinzipien einer REST-Architektur folgt. REST ist die Abkürzung für „Representational State Transfer“ und bezeichnet verschiedene Regeln und Richtlinien, die festlegen, wie eine Web-API aufgebaut sein sollte.

Was ist REST?

REST ist eine Sammlung an Prinzipien, an die man sich als Entwickler halten kann. Dabei geht es darum, wie Ressourcen/Daten repräsentiert werden, wie diese übers Netzwerk übertragen und dargestellt werden und wie sie upgedatet werden können.

Schlüssel Prinzipien von REST:

- a) Ressourcenorientiert (resource-oriented)
- b) Zustandslos (stateless): Clients stellen requests an Server. Server speichern keine Informationen über bisherige requests. Nach dem Request ist vor dem Request.
- c) HTTP Methoden: GET, POST, PUT, DELETE, PATCH.
- d) Einheitliche, konsistente URL-Syntax für die Ressourcen

HTTP-Methoden

GET: Retrieve data from an endpoint.

POST: Send data and create a new resource.

PUT: Update an existing resource.

DELETE: Remove a resource.

PATCH: Partial updates to a resource.

Wenn eine Client-Anfrage über eine RESTful API gestellt wird, überträgt sie die request Daten an den Server. Diese Informationen oder Darstellungen werden in einem von mehreren Formaten über **HTTP** bereitgestellt: JSON (Javascript Object Notation), HTML, XML, Python, PHP oder als einfacher Text. JSON ist hierbei die beliebteste Methode, weil es von Menschen als auch von Maschinen gelesen werden kann.

Vorteile von RESTful APIs:

1. Einfache Nutzung durch HTTP-Standardmethoden wie GET, POST, etc.
2. Ressourcenorientiert und daher nutzbar von vielen verschiedenen Geräten und Betriebssystemen.
3. Bestehende Frameworks stellen Tools zur Authentifizierung und zur Kommunikation mit vielen Nutzern zur Verfügung.
4. Bestehende Bibliotheken: Entwicklung einer RESTful-API zum Beispiel in python mit Flask
(pip install Flask)
5. Öffentliches, einfaches Interface zur Interaktion mit der API
6. Trennung von Anwendungs-Logik, Datentransfer übers Internet und Darstellung

7. Skalierbarkeit: viele gleichzeitige Anfragen beantwortbar. Systeme mit steigenden Anforderungen nach oben hin erweiterbar.



Programmierung einer Flask RESTful API in python:

<https://www.youtube.com/watch?v=z3YMz-Gocmw>



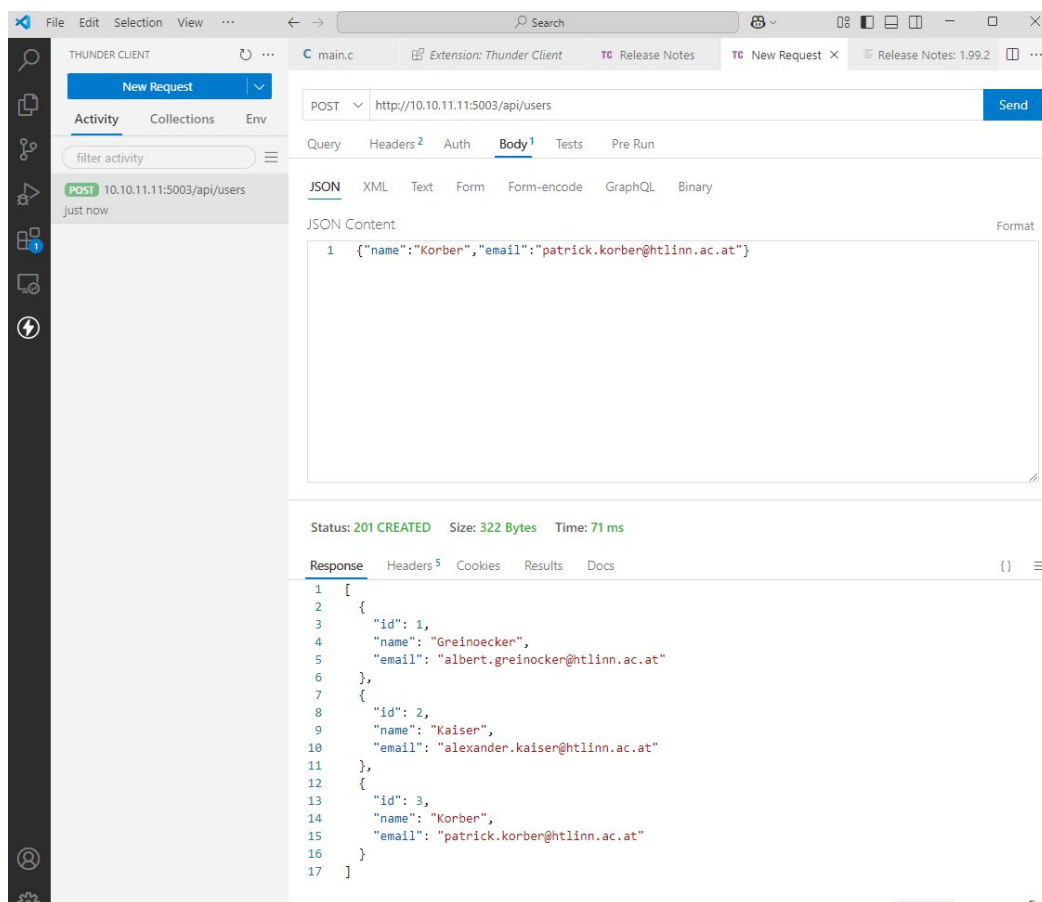
Der Flask Server von api.py aus dem Video läuft bereits auf dem Server 10.10.11.11 und ist unter dem Port 5003 erreichbar. Die API erreichst du also mit der URL

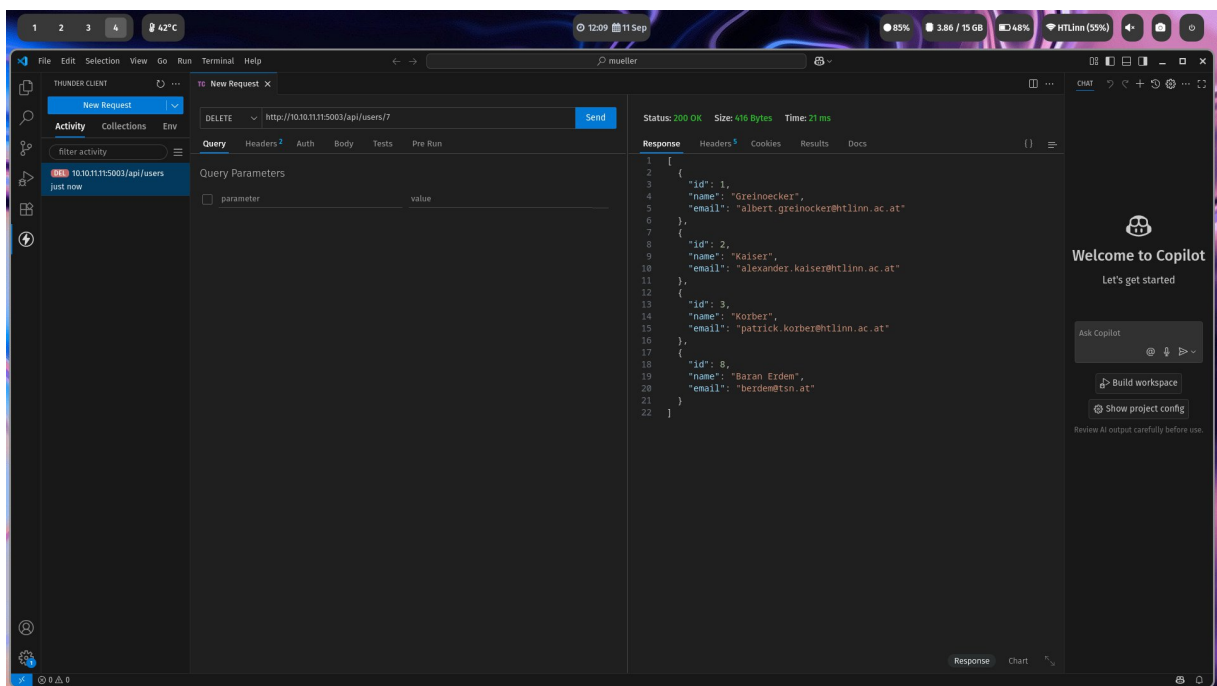
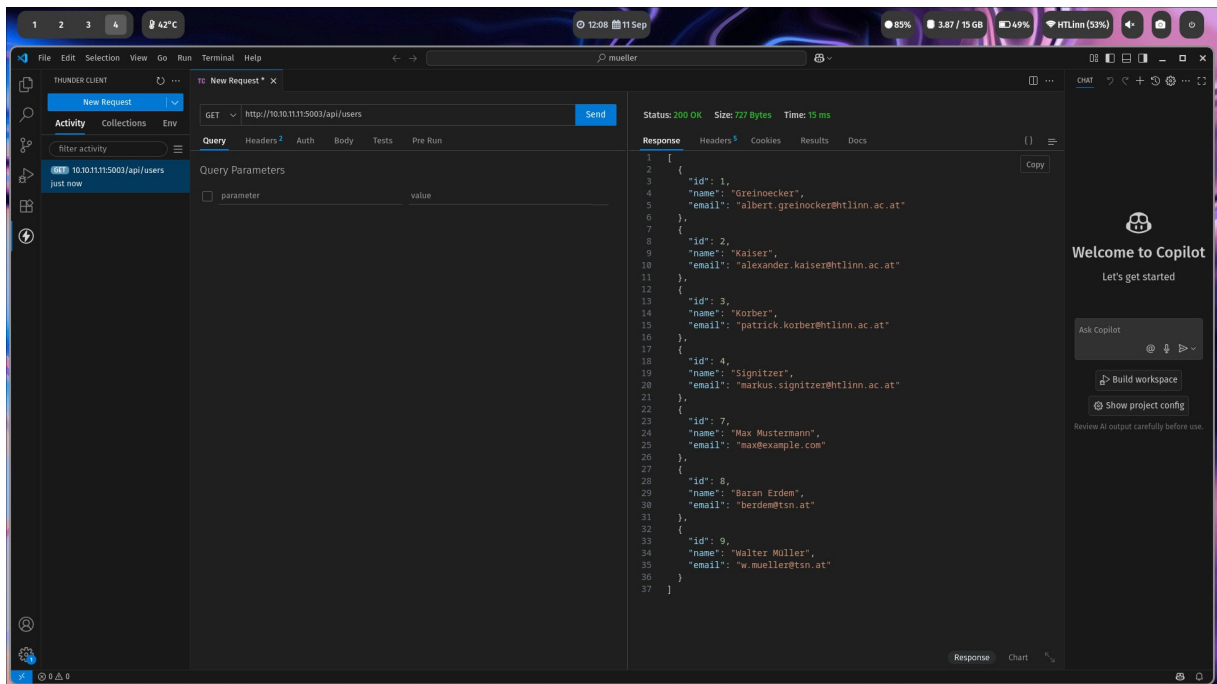
<http://10.10.11.11:5003/api/users>

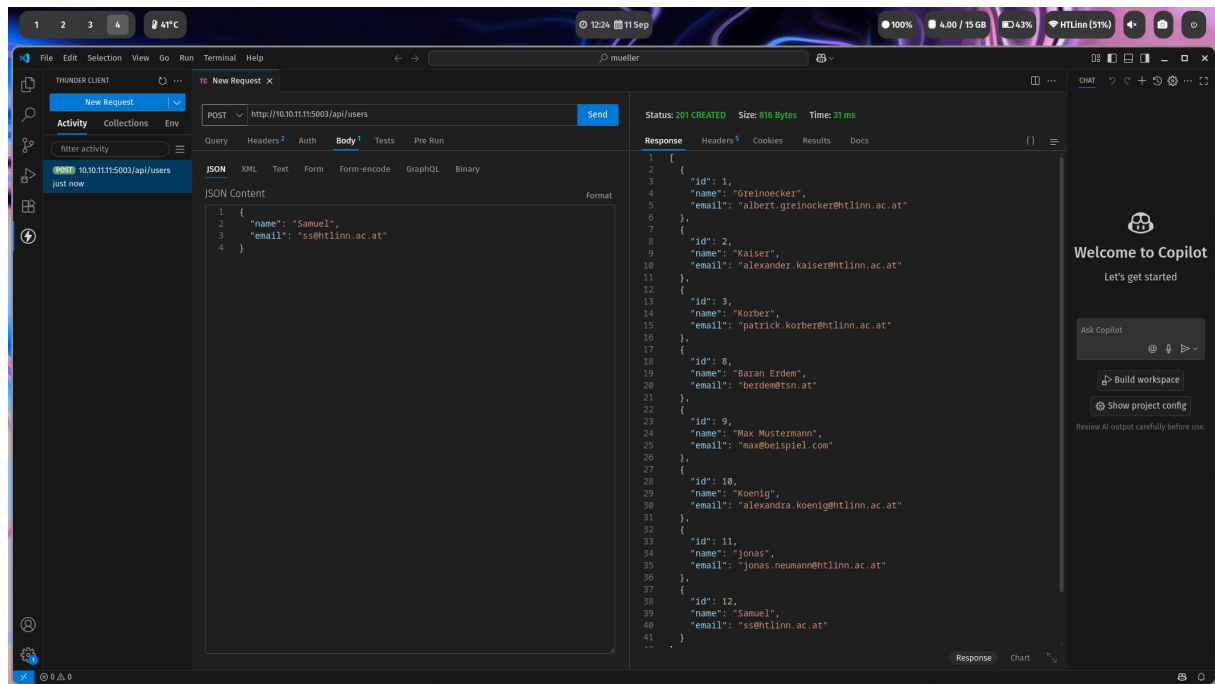
RESTful API nutzen

Installiere die **Thunderclient** extension in VS-Code und öffne sie (Blitz Symbol in der linken Leiste).

Klicke auf „New Request“ und führe GET, POST und DELETE requests durch! Dokumentiere die Ergebnisse, in dem du ein Video Tutorial erstellst mit der „Aufzeichnen“ Funktion des Windows-Snipping Tools.







Weitere Aufgabenstellungen:

Bitte wähle nach eigenem Geschmack eine oder mehrere der Programmieraufgaben aus und dokumentiere diese hier im Arbeitsauftrag.

a) Programmierung eines RESTful Clients:

Erstelle ein Python-Programm mit der Request-Bibliothek mit dem du am obigen Server

- Einen Datensatz anlegst
- Einen Datensatz ausgibst
- Datensätze suchst

Hinweise dazu findest du z.B. im RealPython Tutorial:

<https://realpython.com/api-integration-in-python/#rest-and-python-consuming-apis>

oder in meiner ChatGPT Recherche:

<https://chatgpt.com/share/68c274c7-a1e4-8012-8ef3-fa8c33006d17>

```
1 2 3 4 46°C 11:28 18 Sep 100% 4.09 / 15 GB 67% HTLinn (51%)

[0] 1:yazi*
.rw r r 4.3M samudev 18 Sep 11:14 AAP11-Skriptum_REST_APIv2_1.docx
.rw r r 1.3k samudev 18 Sep 11:25 client.py
.rw r r 1.3k samudev 18 Sep 11:25 README.md
.rw r r 20 samudev 18 Sep 11:25 requirements.txt
.rw r r 4.3k samudev 18 Sep 11:11 weather.py

~/school/fsst/mueller
> python client.py
Usage:
python3 client.py list
python3 client.py get <id>
python3 client.py create <name> <email>

~/school/fsst/mueller
> python client.py list
[[{"id": 1, "name": "Greinoecker", "email": "albert.greinoecker@htlinn.ac.at"}, {"id": 2, "name": "Kaiser", "email": "alexander.kaiser@htlinn.ac.at"}, {"id": 3, "name": "Korber", "email": "patrick.korber@htlinn.ac.at"}, {"id": 8, "name": "Baran Erdem", "email": "berdem@tsn.at"}, {"id": 11, "name": "Jonas", "email": "jonas.neumann@htlinn.ac.at"}, {"id": 12, "name": "Samuel", "email": "ss@htlinn.ac.at"}, {"id": 17, "name": "Sandro Haslwagner", "email": "sahaslwagner@tsn.at"}, {"id": 18, "name": "Max2", "email": "mglatz@tsn.at"}, {"id": 19, "name": "Glaser", "email": "loglaser@tsn.at"}, {"id": 22, "name": "Samuel Falkner", "email": "samfalkner@tsn.at"}, {"id": 23, "name": "Junker", "email": "fjunker@tsn.at"}, {"id": 24, "name": "Dominik Heiss", "email": "domheiss@tsn.at"}, {"id": 25, "name": "Lorenz", "email": "Loglaser@tsn.at"}, {"id": 26, "name": "Lisa", "email": "likarbon@tsn.at"}]]

~/school/fsst/mueller
>

13 #!/usr/bin/env python3
14
15 import sys
16 import requests
17
18 BASE_URL = "http://10.10.11.11:5003/api/users"
19
20 def usage() -> None:
21     print("Usage:")
22     print(" python3 client.py list")
23     print(" python3 client.py get <id>")
24     print(" python3 client.py create <name> <email>")
25
26 def main(argv: list[str]) -> int:
27     if not argv:
28         usage()
29         return 2
30     cmd = argv[0]
31
32     try:
33         if cmd == "list":
34             resp = requests.get(BASE_URL, timeout=10)
35             resp.raise_for_status()
36             print(resp.json())
37             return 0
38         elif cmd == "get":
39             if len(argv) < 2:
40                 usage()
41                 return 2
42             user_id = argv[1]
43             resp = requests.get(f"{BASE_URL}/{user_id}", timeout=10)
44             if resp.status_code == 404:
45                 print(f"error: 'Not Found', 'id': user_id")
46                 return 1
47             resp.raise_for_status()
48             print(resp.json())
49             return 0
50         elif cmd == "create":
51             if len(argv) < 3:
52                 usage()
53                 return 2
54             name = argv[1]
55             email = argv[2]
56             resp = requests.post(BASE_URL, json={"name": name, "email": email}, timeout=10)
57             resp.raise_for_status()
```

b) Realisierung des Servers aus dem Youtube-Tutorial am eigenen PC:

Siehe Git:

<https://github.com/gitdagray/python-flask-rest-api>

Dokumentiere dabei deine Schritte und Probleme.

```
1 2 3 4 46°C 12:00 18-Sep-25 100% 3.98 / 15 GB 53% HTLinn (37%)

[0] 1:python*
Successfully installed Flask-RESTful-0.3.10 Flask-SQLAlchemy-3.1.1 aniso8601-10.0.1 blinker-1.9.0 click-8.2.1 flask-3.1.2 greenlet-3.2.4 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.2 pytz-2025.2 six-1.17.0 sqlalchemy-2.0.43 typing-extensions-4.15.0 Werkzeug-3.1.3

~/school/fsst/mueller/python-flask-rest-api main* 6s
> ls
drwxr-xr-x samudev 18 Sep 11:42 .git
drwxr-xr-x samudev 18 Sep 11:34 __pycache__
drwxr-xr-x samudev 18 Sep 11:32 instance
drwxr-xr-x samudev 18 Sep 11:48 venv
.rw r r 6 samudev 18 Sep 11:32 .gitignore
.rw r r 2.4k samudev 18 Sep 11:32 api.py
.rw r r 69 samudev 18 Sep 11:32 create_db.py
.rw r r 1.6k samudev 18 Sep 11:32 README.md
.rw r r 38 samudev 18 Sep 11:48 requirements.txt

~/school/fsst/mueller/python-flask-rest-api main* 6s
> python create_db.py

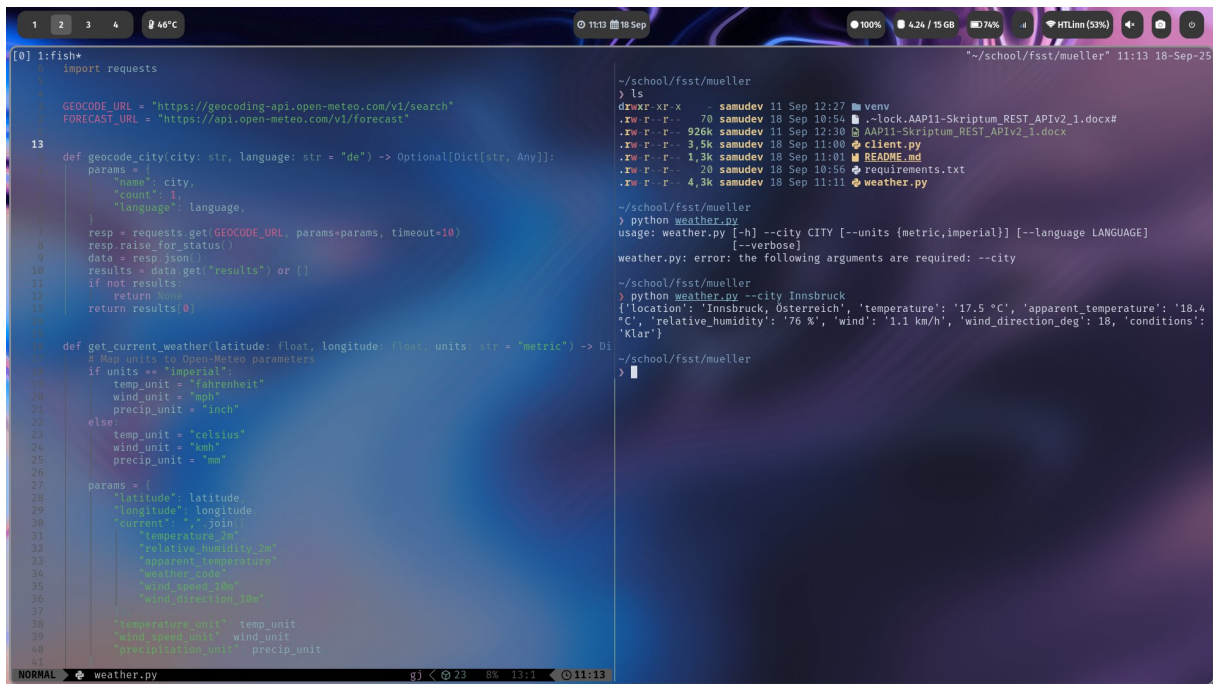
~/school/fsst/mueller/python-flask-rest-api main* 6s
> python api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 142-218-609
^C

~/school/fsst/mueller/python-flask-rest-api main* 8m 39s
> python api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 142-218-609
127.0.0.1 - - [18/Sep/2025 11:59:17] "GET /api/users HTTP/1.1" 308 -
127.0.0.1 - - [18/Sep/2025 11:59:17] "GET /api/users/ HTTP/1.1" 200 -

1 File Edit Selection ... mueller
Welcome TO New Request X
Query Headers Auth Body Tests Pre Run
Query Parameters
parameter value
Status: 200 OK Size: 96 Bytes Time: 29 ms
Response Headers Cookies Results Docs
1 [
2 {
3   "id": 2,
4   "name": "Dave",
5   "email": "dave@davegray.codes"
6 }
7 ]
```

Probleme beim installieren von requirements, sonst ohne Probleme, die Libraries waren für eine ältere Python-Version gemacht, ich habe alle unnötigen Libraries entfernt und die vorgegebenen Versionen entfernt, nach Erstellen einer neuen venv und pip install -r requirements.txt hat alles funktioniert

c) Verwende eine andere öffentliche RestAPI für ein Service deiner Wahl



```
[0] 1:fish*
import requests

GEOCODE_URL = "https://geocoding-api.open-meteo.com/v1/search"
FORECAST_URL = "https://api.open-meteo.com/v1/forecast"

13 def geocode_city(city: str, language: str = "de") -> Optional[Dict[str, Any]]:
    params = {
        "name": city,
        "count": 1,
        "language": language,
    }
    resp = requests.get(GEOCODE_URL, params=params, timeout=10)
    resp.raise_for_status()
    data = resp.json()
    results = data.get('results') or []
    if not results:
        return None
    return results[0]

14
15
16 def get_current_weather(latitude: float, longitude: float, units: str = "metric") -> Dict:
17     # Map units to Open-Meteo parameters
18     if units == "imperial":
19         temp_unit = "fahrenheit"
20         wind_unit = "mph"
21         precip_unit = "inch"
22     else:
23         temp_unit = "celsius"
24         wind_unit = "kmh"
25         precip_unit = "mm"
26
27     params = {
28         "latitude": latitude,
29         "longitude": longitude,
30         "current": "true",
31         "temperature_2d": "true",
32         "relative_humidity_2d": "true",
33         "apparent_temperature": "true",
34         "weather_code": "true",
35         "wind_speed_10m": "true",
36         "wind_direction_10m": "true",
37         "precipitation_units": temp_unit,
38         "wind_speed_unit": wind_unit,
39         "precipitation_unit": precip_unit
40     }
41
42     resp = requests.get(FORECAST_URL, params=params, timeout=10)
43     resp.raise_for_status()
44     data = resp.json()
45     return data.get('current_weather')

46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
~/school/fsst/mueller
> ls
drwxr-xr-x  70 samdev 11 Sep 12:27 .
-rw-r--r--  70 samdev 18 Sep 10:54 .lock.AAP11-Skriptum_REST_APIv2_1.docx#
-rw-r--r-- 926k samdev 11 Sep 12:30 AAP11-Skriptum_REST_APIv2_1.docx
-rw-r--r  3,9k samdev 18 Sep 11:00 client.py
-rw-r--r  1,3k samdev 18 Sep 11:01 README.md
-rw-r--r   20 samdev 18 Sep 10:56 requirements.txt
-rw-r--r   4,3k samdev 18 Sep 11:11 weather.py

~/school/fsst/mueller
> python weather.py
usage: weather.py [-h] --city CITY [--units {metric,imperial}] [--language LANGUAGE]
                  [--verbose]
weather.py: error: the following arguments are required: --city

~/school/fsst/mueller
> python weather.py --city Innsbruck
{'location': 'Innsbruck, Österreich', 'temperature': '17.5 °C', 'apparent_temperature': '18.4 °C', 'relative_humidity': '76 %', 'wind': '1.1 km/h', 'wind_direction_deg': 18, 'conditions': 'Klar'}
```

Dokumentiere dabei deine Schritte und Probleme.

GEOCODE_URL = "https://geocoding-api.open-meteo.com/v1/search"

FORECAST_URL = "https://api.open-meteo.com/v1/forecast"

Chatgpt hat mir diese empfohlen, keine Probleme

Kontrollfragen zum Verständnis:

- Welche Vorteile haben RESTful APIs? Standard http methoden
- Erkläre eine den Aufbau einer RESTful URL zur Abfrage des zweiten Datensatzes. Get /users/2
- Welche im allgemeinen Features von HTTPS können bei RESTful APIs „gratis“ mit verwendet werden? Verschlüsselung, server verifizierung, vertraulichkeit
- Was bedeutet stateless? Speichert keine zustände

Welche Vorteile bietet dies in Bezug auf die Serverarchitektur? Einfache skalierung

- Zu obigem Web-Tutorial:
 - o Wie funktioniert ein Flask-Webserver?
Wann wird der gestartet? Läuft der immer? Wie erfolgt die Behandlung einzelner URLs? Web Server, bei python app.py, läuft solange die python datei läuft, mit routen oder eigenen Befehlen
 - o Aus welcher Bibliothek stammt SQLAlchemy?
Was leistet diese Bibliothek? Python bibliothek, arbeitet mit Datenbanken wie MySQL
 - o SCHWIERIG: Was ist ein Decorator und was macht der @app.route() bzw. @marshal_with() Decorator ist eine Funktion die eine Funktion erweitert, @app.route verbindet eine URL mit einer Route, marshal_with formatiert automatisch

Lerntagebuch:

Was habe ich wann und wie lange gemacht?

11.9 thunderclient, eigenen Restclient schreiben

18.9 eigenen Restclient fertigen, öffentliche API benutzen

Was hat Probleme bereitet? Typ festlegen (thunderclient)

Wobei habe ich am meisten gelernt? bei öffentlichen API's benutzen

Quellen:

Aufbauend auf den Lesetext von <https://www.redhat.com/de/topics/api/what-is-a-rest-api>

©KaisA Mit deepseek und chatgpt 4.0 überarbeitet

©MuelW Erweiterung um Programmieraufgaben und Verständnisfragen